

Introduction to R Workshop

Getting Started, Data Visualization, and Regression

Muji Chughtai

2026-07-16

Getting Started

Introduction to R and RStudio

Introduction to R and RStudio

- R is a free software environment for statistical computing and graphics that can compile and run on a variety of platforms.

Introduction to R and RStudio

- R is a free software environment for statistical computing and graphics that can compile and run on a variety of platforms.
 - To download R, go to <https://cran.r-project.org>.

Introduction to R and RStudio

- R is a free software environment for statistical computing and graphics that can compile and run on a variety of platforms.
 - To download R, go to <https://cran.r-project.org>.
- For a more user-friendly interface, RStudio is a popular integrated development environment (IDE) for R.

Introduction to R and RStudio

- R is a free software environment for statistical computing and graphics that can compile and run on a variety of platforms.
 - To download R, go to <https://cran.r-project.org>.
- For a more user-friendly interface, RStudio is a popular integrated development environment (IDE) for R.
- RStudio also allows for powerful document creation using Markdown, which can generate reports, presentations, and dashboards (this presentation was made using a Quarto Markdown in RStudio).

Introduction to R and RStudio

- R is a free software environment for statistical computing and graphics that can compile and run on a variety of platforms.
 - To download R, go to <https://cran.r-project.org>.
- For a more user-friendly interface, RStudio is a popular integrated development environment (IDE) for R.
- RStudio also allows for powerful document creation using Markdown, which can generate reports, presentations, and dashboards (this presentation was made using a Quarto Markdown in RStudio).
 - To download RStudio, go to <https://www.rstudio.com/products/rstudio/download>.

Getting Familiar with the RStudio Interface

Getting Familiar with the RStudio Interface

There are 4 main panels in the RStudio interface:

Getting Familiar with the RStudio Interface

There are 4 main panels in the RStudio interface:

1. **Source Panel (Top Left):** This is where you can write and edit your R scripts and markdown files. Toggling between tabs allows you to manage multiple files at once.

Getting Familiar with the RStudio Interface

There are 4 main panels in the RStudio interface:

1. **Source Panel (Top Left):** This is where you can write and edit your R scripts and markdown files. Toggling between tabs allows you to manage multiple files at once.
2. **Console Panel (Bottom Left):** This is where you can directly type and execute R commands. When you run code from the Source panel, the output will also appear here.

Getting Familiar with the RStudio Interface

There are 4 main panels in the RStudio interface:

1. **Source Panel (Top Left):** This is where you can write and edit your R scripts and markdown files. Toggling between tabs allows you to manage multiple files at once.
2. **Console Panel (Bottom Left):** This is where you can directly type and execute R commands. When you run code from the Source panel, the output will also appear here.
3. **Environment/History Panel (Top Right):** The Environment tab shows the objects (variables, data frames, functions, etc.) currently in your R session.

Getting Familiar with the RStudio Interface

There are 4 main panels in the RStudio interface:

1. **Source Panel (Top Left):** This is where you can write and edit your R scripts and markdown files. Toggling between tabs allows you to manage multiple files at once.
2. **Console Panel (Bottom Left):** This is where you can directly type and execute R commands. When you run code from the Source panel, the output will also appear here.
3. **Environment/History Panel (Top Right):** The Environment tab shows the objects (variables, data frames, functions, etc.) currently in your R session.
4. **Files/Plots/Packages/Help/Viewer Panel (Bottom Right):** This panel has multiple tabs for managing files, viewing plots, installing and loading packages, accessing R help documentation, and displaying web content.

Using Markdown

Using Markdown

- R Markdown is a simple formatting syntax for authoring documents. It allows you to combine text, code, and output in a single document. You can create reports, presentations, and dashboards using R Markdown.

Using Markdown

- R Markdown is a simple formatting syntax for authoring documents. It allows you to combine text, code, and output in a single document. You can create reports, presentations, and dashboards using R Markdown.
- In R Markdown, you write code in **chunks** that are enclosed by triple backticks and curly braces.

Using Markdown

- R Markdown is a simple formatting syntax for authoring documents. It allows you to combine text, code, and output in a single document. You can create reports, presentations, and dashboards using R Markdown.
- In R Markdown, you write code in **chunks** that are enclosed by triple backticks and curly braces.
 - You can create R chunks using the green “c” button in the RStudio toolbar or by typing in three backticks followed by `{r}`.

Using Markdown

- R Markdown is a simple formatting syntax for authoring documents. It allows you to combine text, code, and output in a single document. You can create reports, presentations, and dashboards using R Markdown.
- In R Markdown, you write code in **chunks** that are enclosed by triple backticks and curly braces.
 - You can create R chunks using the green “c” button in the RStudio toolbar or by typing in three backticks followed by `{r}`.
 - If you want to write in other languages in the code chunk, you can specify the language in the curly braces (e.g., `{python}`, `{bash}`, etc.).

Using Markdown

- R Markdown is a simple formatting syntax for authoring documents. It allows you to combine text, code, and output in a single document. You can create reports, presentations, and dashboards using R Markdown.
- In R Markdown, you write code in **chunks** that are enclosed by triple backticks and curly braces.
 - You can create R chunks using the green “c” button in the RStudio toolbar or by typing in three backticks followed by `{r}`.
 - If you want to write in other languages in the code chunk, you can specify the language in the curly braces (e.g., `{python}`, `{bash}`, etc.).
- I suggest you do your analyses in R Markdown documents, as it allows you to keep your code and results together in a reproducible format that can be easily exported to various formats (HTML, PDF, Word, etc.)!

Staying Organized

Staying Organized

- One of the biggest issues that people face when programming (in R) is keeping track of code, files, and output.

Staying Organized

- One of the biggest issues that people face when programming (in R) is keeping track of code, files, and output.
- To help with this, I encourage you to create a dedicate **RProject** for each project you work on. This will keep all of your files in one folder, and within this folder you can have subfolders for **Data**, **Analyses**, **Figures**, etc.

Staying Organized

- One of the biggest issues that people face when programming (in R) is keeping track of code, files, and output.
- To help with this, I encourage you to create a dedicate **RProject** for each project you work on. This will keep all of your files in one folder, and within this folder you can have subfolders for **Data**, **Analyses**, **Figures**, etc.
- The **here** package is a great tool to help you manage file paths in your R projects. It allows you to use relative paths instead of absolute paths, which makes your code more portable and easier to share.

Data Visualization

Creating a Directory

Creating a Directory

Let's start programming in R by creating a small project where we will visualize and analyze some data from using data from the World Bank in 2016.

Creating a Directory

Let's start programming in R by creating a small project where we will visualize and analyze some data from using data from the World Bank in 2016.

- Open RStudio and create a new project (and corresponding folder) by navigating from the menu to **File > New Project > New Directory > New Project**, and name your project whatever you like and save it in a location you prefer.

Creating a Directory

Let's start programming in R by creating a small project where we will visualize and analyze some data from using data from the World Bank in 2016.

- Open RStudio and create a new project (and corresponding folder) by navigating from the menu to **File > New Project > New Directory > New Project**, and name your project whatever you like and save it in a location you prefer.
- Within that project folder, create a new R Markdown document by navigating to **File > New File > R Markdown**, and you can do this from either the menu or the bottom right panel. Name it whatever you like!

Installing and Loading Packages

Installing and Loading Packages

- Let's load in two packages that we will use for this workshop: `tidyverse` and `here`.

Installing and Loading Packages

- Let's load in two packages that we will use for this workshop: **tidyverse** and **here**.
 - **Tidyverse** is a collection of R packages that are designed for data science. It includes packages for data manipulation, visualization, and modeling.

Installing and Loading Packages

- Let's load in two packages that we will use for this workshop: **tidyverse** and **here**.
 - **Tidyverse** is a collection of R packages that are designed for data science. It includes packages for data manipulation, visualization, and modeling.
 - **Here** is a package that helps you manage file paths in your R projects.

Installing and Loading Packages

- Let's load in two packages that we will use for this workshop: **tidyverse** and **here**.
 - **Tidyverse** is a collection of R packages that are designed for data science. It includes packages for data manipulation, visualization, and modeling.
 - **Here** is a package that helps you manage file paths in your R projects.
- To load in packages in R you use the **library()** function:

Installing and Loading Packages

- Let's load in two packages that we will use for this workshop: **tidyverse** and **here**.
 - **Tidyverse** is a collection of R packages that are designed for data science. It includes packages for data manipulation, visualization, and modeling.
 - **Here** is a package that helps you manage file paths in your R projects.
- To load in packages in R you use the **library()** function:

Installing and Loading Packages

- Let's load in two packages that we will use for this workshop: **tidyverse** and **here**.
 - **Tidyverse** is a collection of R packages that are designed for data science. It includes packages for data manipulation, visualization, and modeling.
 - **Here** is a package that helps you manage file paths in your R projects.
- To load in packages in R you use the `library()` function:

```
library(tidyverse)
library(here)
```

Installing and Loading Packages

- This code won't work if you don't have the packages installed. To install them, you can use the `install.packages()` function, and you should put this code chunk above the `library()` code chunk in your R Markdown document:

Installing and Loading Packages

- This code won't work if you don't have the packages installed. To install them, you can use the `install.packages()` function, and you should put this code chunk above the `library()` code chunk in your R Markdown document:

Installing and Loading Packages

- This code won't work if you don't have the packages installed. To install them, you can use the `install.packages()` function, and you should put this code chunk above the `library()` code chunk in your R Markdown document:

```
install.packages("tidyverse", repos = "https://cloud.r-project.org/")  
install.packages("here", repos = "https://cloud.r-project.org/")
```

Installing and Loading Packages

- This code won't work if you don't have the packages installed. To install them, you can use the `install.packages()` function, and you should put this code chunk above the `library()` code chunk in your R Markdown document:

```
install.packages("tidyverse", repos = "https://cloud.r-project.org/")  
install.packages("here", repos = "https://cloud.r-project.org/")
```

- Newer versions of RStudio will also allow you to download packages from the top menu if you don't have them.

Installing and Loading Packages

- This code won't work if you don't have the packages installed. To install them, you can use the `install.packages()` function, and you should put this code chunk above the `library()` code chunk in your R Markdown document:

```
install.packages("tidyverse", repos = "https://cloud.r-project.org/")  
install.packages("here", repos = "https://cloud.r-project.org/")
```

- Newer versions of RStudio will also allow you to download packages from the top menu if you don't have them.
- You can also directly install packages from the console by typing in the same code above (when you run a chunk it will run in the console).

Installing and Loading Packages

Installing and Loading Packages

- You can make comments in your code using the `#` symbol within a chunk. Comments are ignored by R when running code, and they are a great way to explain what your code is doing:

Installing and Loading Packages

- You can make comments in your code using the `#` symbol within a chunk. Comments are ignored by R when running code, and they are a great way to explain what your code is doing:

Installing and Loading Packages

- You can make comments in your code using the `#` symbol within a chunk. Comments are ignored by R when running code, and they are a great way to explain what your code is doing:

```
# loading in required packages  
library(tidyverse)  
library(here)
```

Installing and Loading Packages

- You can make comments in your code using the `#` symbol within a chunk. Comments are ignored by R when running code, and they are a great way to explain what your code is doing:

```
# loading in required packages  
library(tidyverse)  
library(here)
```

- If you use the `#` symbol outside of a chunk, it will be treated as a header in your R Markdown document. The more `#` symbols you use, the smaller the header will be.

Installing and Loading Packages

- You can make comments in your code using the `#` symbol within a chunk. Comments are ignored by R when running code, and they are a great way to explain what your code is doing:

```
# loading in required packages  
library(tidyverse)  
library(here)
```

- If you use the `#` symbol outside of a chunk, it will be treated as a header in your R Markdown document. The more `#` symbols you use, the smaller the header will be.
- These headings will transfer into the output document that you render for easy navigation in the html, pdf, or word document.

Loading in the Data

Loading in the Data

- Let's load in the data from this project, which is a csv file that contains data from the World Bank in 2016.

Loading in the Data

- Let's load in the data from this project, which is a csv file that contains data from the World Bank in 2016.
- The data and codebook is available on my github at <https://github.com/mujtabachughtai/2026-R-Workshop>.

Loading in the Data

- Let's load in the data from this project, which is a csv file that contains data from the World Bank in 2016.
- The data and codebook is available on my github at <https://github.com/mujtabachughtai/2026-R-Workshop>.
- Download the data (called `WB.2016.csv`), create a `Data` subfolder in this project's directory, and save the data in that folder.

Loading in the Data

- Then you should be able to load in the data using the `here` package, where you can name your new dataframe `WorldBank`

Loading in the Data

- Then you should be able to load in the data using the `here` package, where you can name your new dataframe `WorldBank`

Loading in the Data

- Then you should be able to load in the data using the `here` package, where you can name your new dataframe `WorldBank`

```
WorldBank <- read.csv(  
  here("Data", "WB.2016.csv"),  
  fileEncoding = "Windows-1252"  
)
```

Loading in the Data

- Then you should be able to load in the data using the `here` package, where you can name your new dataframe `WorldBank`

```
WorldBank <- read.csv(  
  here("Data", "WB.2016.csv"),  
  fileEncoding = "Windows-1252"  
)
```

- Note: You normally don't need the `fileEncoding` argument, but this dataset has some special characters that require it.

Loading in the Data

- Then you should be able to load in the data using the `here` package, where you can name your new dataframe `WorldBank`

```
WorldBank <- read.csv(  
  here("Data", "WB.2016.csv"),  
  fileEncoding = "Windows-1252"  
)
```

- Note: You normally don't need the `fileEncoding` argument, but this dataset has some special characters that require it.
- You should now be able to view your data in the `Environment` tab in the top right panel of RStudio.

Getting a Feel for the Data

Getting a Feel for the Data

- When you look at the dataframe in RStudio, you can see what variables we have to explore.

Getting a Feel for the Data

- When you look at the dataframe in RStudio, you can see what variables we have to explore.
- For this workshop, we will be focusing on the following variables currently in the `WorldBank` dataframe:

Getting a Feel for the Data

- When you look at the dataframe in RStudio, you can see what variables we have to explore.
- For this workshop, we will be focusing on the following variables currently in the `WorldBank` dataframe:
 - **Country:** The name of the country

Getting a Feel for the Data

- When you look at the dataframe in RStudio, you can see what variables we have to explore.
- For this workshop, we will be focusing on the following variables currently in the `WorldBank` dataframe:
 - **Country:** The name of the country
 - **Code:** A 3-letter code for each country

Getting a Feel for the Data

- When you look at the dataframe in RStudio, you can see what variables we have to explore.
- For this workshop, we will be focusing on the following variables currently in the `WorldBank` dataframe:
 - **Country:** The name of the country
 - **Code:** A 3-letter code for each country
 - **Population:** The population of the country

Getting a Feel for the Data

- When you look at the dataframe in RStudio, you can see what variables we have to explore.
- For this workshop, we will be focusing on the following variables currently in the `WorldBank` dataframe:
 - **Country:** The name of the country
 - **Code:** A 3-letter code for each country
 - **Population:** The population of the country
 - **GNI:** Gross National Income per capita

Getting a Feel for the Data

- When you look at the dataframe in RStudio, you can see what variables we have to explore.
- For this workshop, we will be focusing on the following variables currently in the `WorldBank` dataframe:
 - **Country:** The name of the country
 - **Code:** A 3-letter code for each country
 - **Population:** The population of the country
 - **GNI:** Gross National Income per capita
- And we're going to make some variables of our own, too.

Getting a Feel for the Data

- Let's take a look at the first few rows of these key variables. We will use the `knitr::kable` and `head` functions to do so.

```
knitr::kable(head(  
  WorldBank[, c("Country", "Code", "Population", "GNI")]))
```

Country	Code	Population	GNI
Afghanistan	AFG	34656032	580
Albania	ALB	2876101	4320
Algeria	DZA	40606052	4360
American Samoa	ASM	55599	NA
Andorra	AND	77281	NA
Angola	AGO	28813463	3450

Getting a Feel for the Data

Getting a Feel for the Data

- It looks like there is some missingness in data, which is something we should keep in mind as we are doing our analyses.

Getting a Feel for the Data

- It looks like there is some missingness in data, which is something we should keep in mind as we are doing our analyses.
- We can check the means of our key variables using the R's built in mean function:

Getting a Feel for the Data

- It looks like there is some missingness in data, which is something we should keep in mind as we are doing our analyses.
- We can check the means of our key variables using the R's built in mean function:

Getting a Feel for the Data

- It looks like there is some missingness in data, which is something we should keep in mind as we are doing our analyses.
- We can check the means of our key variables using the R's built in mean function:

```
# mean of Population  
mean(WorldBank$Population, na.rm = TRUE)
```

```
[1] 34331920
```

Getting a Feel for the Data

- It looks like there is some missingness in data, which is something we should keep in mind as we are doing our analyses.
- We can check the means of our key variables using the R's built in mean function:

```
# mean of Population  
mean(WorldBank$Population, na.rm = TRUE)
```

```
[1] 34331920
```

```
# mean of GNI  
mean(WorldBank$GNI, na.rm = TRUE)
```

```
[1] 12794.73
```

Getting a Feel for the Data

Getting a Feel for the Data

- We can also look at the distributions of these variables using `geom_density` from `ggplot`:

Getting a Feel for the Data

- We can also look at the distributions of these variables using `geom_density` from `ggplot`:

Getting a Feel for the Data

- We can also look at the distributions of these variables using `geom_density` from `ggplot`:

```
WorldBank %>%  
  ggplot(aes(x = Population)) +  
  geom_density()
```

Getting a Feel for the Data

- We can also look at the distributions of these variables using `geom_density` from `ggplot`:

```
WorldBank %>%  
  ggplot(aes(x = Population)) +  
  geom_density()
```

- This can equivalently be written as

Getting a Feel for the Data

- We can also look at the distributions of these variables using `geom_density` from `ggplot`:

```
WorldBank %>%  
  ggplot(aes(x = Population)) +  
  geom_density()
```

- This can equivalently be written as

Getting a Feel for the Data

- We can also look at the distributions of these variables using `geom_density` from `ggplot`:

```
WorldBank %>%  
  ggplot(aes(x = Population)) +  
  geom_density()
```

- This can equivalently be written as

```
ggplot(aes(x = Population), data = WorldBank) +  
  geom_density()
```

Getting a Feel for the Data

- `ggplot` objects add layers to the plot using the `+` operator, and you can add as many layers as you want.

Getting a Feel for the Data

- `ggplot` objects add layers to the plot using the `+` operator, and you can add as many layers as you want.
- Let's make this plot cleaner by changing the theme and adding title and axis labels:

Getting a Feel for the Data

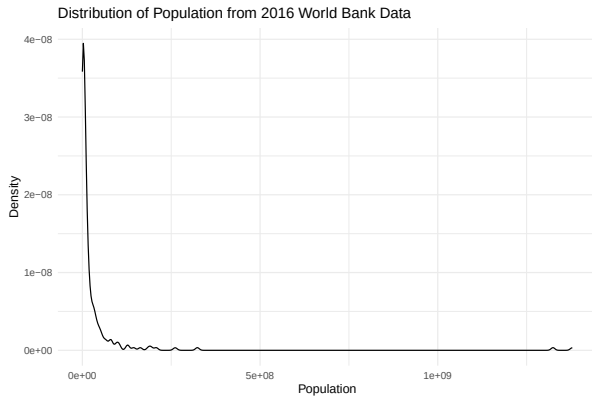
- `ggplot` objects add layers to the plot using the `+` operator, and you can add as many layers as you want.
- Let's make this plot cleaner by changing the theme and adding title and axis labels:

Getting a Feel for the Data

- `ggplot` objects add layers to the plot using the `+` operator, and you can add as many layers as you want.
- Let's make this plot cleaner by changing the theme and adding title and axis labels:

```
WorldBank %>%  
  ggplot(aes(x = Population)) +  
  geom_density() +  
  theme_minimal() +  
  labs(  
    title = "Distribution of Population from 2016 World Bank Data",  
    x = "Population",  
    y = "Density"  
  )
```

Getting a Feel for the Data



Getting a Feel for the Data

- The plot looks great, but the data is very heavily right skewed:

Getting a Feel for the Data

- The plot looks great, but the data is very heavily right skewed:
 - This is because the vast majority of countries that have small populations, while there are a few that have very large ones.

Getting a Feel for the Data

- The plot looks great, but the data is very heavily right skewed:
 - This is because the vast majority of countries that have small populations, while there are a few that have very large ones.
- We can log-transform the data to make it more normally distributed, which is a common technique in statistics. This will also help us with our regression analyses later on.

Theorem

For all real numbers x and y , we have $(x + y)^2 = x^2 + 2xy + y^2 \therefore$

Academic Environments

Use `::: {.block}`, `::: {.alertblock}`, `::: {.exampleblock}` for Pandoc-native Beamer blocks. Use raw LaTeX (````\{=latex\}`) for the custom orange/gray environments.

Definition

A **group** $(G, \cdot)$ is a set with a binary operation satisfying closure, associativity, identity, and invertibility.

Proof

By hypothesis every element of $H \leq G$ satisfies $g^n = e$ for some n . Therefore H is generated by g^k for the smallest such k . $\square$

Important

This theorem requires the axiom of choice for infinite collections.

More Environments

Example

The integers $(\mathbb{Z}, +)$ form an abelian group under addition.

Lists and Structure

- First point with important information

Lists and Structure

- First point with important information
- Second point to demonstrate spacing

Lists and Structure

- First point with important information
- Second point to demonstrate spacing
- Third point showing the style

Lists and Structure

- First point with important information
 - Second point to demonstrate spacing
 - Third point showing the style
1. Numbered lists also work well

Lists and Structure

- First point with important information
 - Second point to demonstrate spacing
 - Third point showing the style
1. Numbered lists also work well
 2. With proper spacing

Lists and Structure

- First point with important information
 - Second point to demonstrate spacing
 - Third point showing the style
1. Numbered lists also work well
 2. With proper spacing
 3. And clean formatting

Two Column Layout

Left Column

- Item one

Right Column

Content on the right side with matching styling

Two Column Layout

Left Column

- Item one
- Item two

Right Column

Content on the right side with matching styling

Two Column Layout

Left Column

- Item one
- Item two
- Item three

Right Column

Content on the right side with matching styling

Code Example

```
# R code with syntax highlighting
library(ggplot2)

ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  theme_minimal()
```

